Topic 15: Searching Algorithms and Complexity Summary

15.1 Linear Search

- Scan each element until target found or end reached
- Time: O(n) worst and average, O(1) best
- Works on unsorted arrays
- Sufficient for small arrays or one-time searches

```
int linearSearch(int* a, int n, int target) {
    for (int i = 0; i < n; i++)
        if (a[i] == target) return i;
    return -1;
}
```

15.2 Binary Search

- Requires sorted array -- compare target with middle element
- If target < mid: search left half; if target > mid: search right half; else found
- Time: O(log n) -- eliminates half the remaining elements each step
- Iterative version preferred (no stack overhead)

```
int binarySearch(int* a, int n, int target) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;  // avoids overflow
        if      (a[mid] == target) return mid;
        else if (a[mid] <  target) lo = mid + 1;
        else                       hi = mid - 1;
    }
    return -1;
}
```

15.3 Complexity Summary Table

- Data Structure | Access | Search | Insert | Delete
- Array | O(1) | O(n) | O(n) | O(n)
- Linked List | O(n) | O(n) | O(1)* | O(1)*
- Stack / Queue | O(n) | O(n) | O(1) | O(1)

- BST (balanced) | O(log n) | O(log n) | O(log n) | O(log n)
- Hash Table | N/A | O(1)** | O(1)** | O(1)**
- Heap | O(1) | O(n) | O(log n) | O(log n)
- * at head with pointer ** average case

15.4 Sorting Complexity Summary

- Bubble Sort: Best O(n), Average O(n^2), Worst O(n^2), Space O(1), Stable
- Selection Sort: Best O(n^2), Average O(n^2), Worst O(n^2), Space O(1), Not Stable
- Insertion Sort: Best O(n), Average O(n^2), Worst O(n^2), Space O(1), Stable
- Merge Sort: Best O(n log n), Average O(n log n), Worst O(n log n), Space O(n), Stable
- Quick Sort: Best O(n log n), Average O(n log n), Worst O(n^2), Space O(log n), Not Stable
- Heap Sort: Best O(n log n), Average O(n log n), Worst O(n log n), Space O(1), Not Stable
- Lower bound for comparison-based sorting: O(n log n) -- cannot do better

15.5 Choosing the Right Data Structure

- Need fast random access? -> Array or vector
- Frequent insert/delete in the middle? -> Linked List
- LIFO behaviour? -> Stack
- FIFO behaviour? -> Queue
- Need sorted data with fast search? -> BST (balanced) or sorted vector + binary search
- Need fastest possible lookup, no order needed? -> Hash Table (unordered_map)
- Need to always access the min or max quickly? -> Heap / Priority Queue
- Represent relationships between entities? -> Graph
